



**LANDMARK METROPOLITAN
UNIVERSITY INSTITUTE**
THE PRIDE OF AFRICA

DATABASE MANAGEMENT & SQL

MODULE 3

Precious Oben, 2025/2026

LANDMARK DIGITAL LESSONS

(+237) 672 339 570 / 694 990 622 | www.landmark.cm



EER DIAGRAMS IN MYSQL WORKBENCH



Identifying Relationships: These occur when the primary key of the related (child) entity includes the primary key of the parent entity. It signifies a strong dependency between the two, where the existence of the child entity is dependent on the parent entity. For example if an 'Enrollment' entity has a composite key that includes StudentID from the student table and CourseID from the course table, the relationship is identifying. This means Enrollment cannot exist without associating both a student and a course. Hence its ordinality is mandatory.

EER DIAGRAMS IN MYSQL WORKBENCH



Non-identifying relationships: These happen when the primary key of the child entity does not include the parent entity's primary key. It implies a weaker dependency, where the child entity can exist independently of the parent entity but still maintains a relationship to it. These relationships can be optional (where the foreign key in the child table that points to the parent table can be null) or mandatory (where the foreign key must always have a value). In an ER diagram, a non-identifying relationship is typically represented with a dotted line.

EER DIAGRAMS IN MYSQL WORKBENCH



Physical ER diagram symbols

	One
	Many
	One (and only one)
	Zero or one
	One or many
	Zero or many

BENEFITS OF DATA MODELING



- **Improved Data Quality and Consistency:** By creating a detailed data model, you enforce standards and rules that help maintain data accuracy and consistency across the database. Data modeling ensures that all data flows follow the defined structures and relationships, reducing data redundancy and errors.
- **Facilitates Communication:** Data models provide a visual representation of data structures and relationships, making it easier for business stakeholders and technical teams to understand the data flows and database structure. This improves communication regarding data requirements and business processes, ensuring everyone is on the same page.
- **Enhances Data Integration:** In complex environments where data from various sources needs to be integrated, data modeling helps identify relationships between disparate data sets. It enables seamless integration and consolidation of data by defining how different data entities relate to each other.
- **Simplifies Database Design:** Data modeling provides a blueprint for database construction, which simplifies the database design process. It allows designers to visualize the structure and assess the impact of changes before implementation, helping to avoid costly revisions and downtime during development.

BENEFITS OF DATA MODELING



- **Supports Business Rules Implementation:** A data model incorporates business rules into the database design, ensuring that these rules are enforced by the database management system. This automatic enforcement of rules helps in maintaining the integrity and validity of the data.
- **Eases Database Maintenance and Scalability:** A well-designed data model is easier to manage and update. It can grow with the business, accommodating new data needs without disrupting existing structures. This scalability and flexibility are crucial for adapting to business changes.
- **Optimizes Performance:** A good data model is structured in a way that optimizes data access and query performance. It minimizes redundancy and organizes data efficiently, leading to faster response times and reduced load on the database server.
- **Compliance and Security:** Effective data modeling can help in implementing data security measures and ensuring compliance with data protection regulations. By defining who can access what data and how they can manipulate it, a data model helps in setting up secure and compliant data handling practices.

DATA MANIPULATION & RETRIEVAL



- **Data manipulation** is done using DML using the following clauses;

INSERT, SELECT, DELETE and UPDATE

- **INSERT:** Used to add new rows to a table.
- **UPDATE:** Modifies existing data within a table.
- **DELETE:** Removes existing rows from a table.
- **SELECT:** Retrieves data from the database

All of these can be used in conjunction with the conditional statement **WHERE**.

DATA MANIPULATION & RETRIEVAL



- **Data retrieval** is a fundamental aspect of SQL, especially when the tables are related through primary and foreign keys.

Recall

- **Primary Key:** A column or a group of columns used to identify a unique row in a table.
- **Foreign Key:** A column or a group of columns in one table that uniquely identifies a row of another table. The foreign key is defined in a second table, but it refers to the primary key or a unique key in the first table.

In order to accurately retrieve data from tables especially related tables, we have to make use of the **JOIN**.

DATA MANIPULATION & RETRIEVAL



SQL JOINS are used to retrieve data from two or more tables based on a relationship between certain columns in these tables. They combine rows from these tables by matching data in key columns, typically where one is a primary key and the other a foreign key.

Types of JOINS

1. INNER JOIN

- **Description:** Returns records that have matching values in both tables.
- **Usage:** Use when you only want to see rows that have corresponding entries in both tables.

2. LEFT JOIN (or LEFT OUTER JOIN)

- **Description:** Returns all records from the left table, and the matched records from the right table. If there is no match, the result is NULL on the side of the right table.
- **Usage:** Use when you want to see all the rows from the left table, including where there are no matches in the right table.

3. RIGHT JOIN (or RIGHT OUTER JOIN)

- **Description:** Returns all records from the right table, and the matched records from the left table. If there is no match, the result is NULL on the side of the left table.
- **Usage:** Use when you want to see all the rows from the right table, including where there are no matches in the left table.

4. FULL JOIN (or FULL OUTER JOIN)

- **Description:** Returns all records when there is a match in either the left or right table. If there is no match, the result is NULL on the side of the table without a match.
- **Usage:** Use when you want to see all rows from both tables, regardless of whether there is a match between the tables.

SELECT column_name(s)

FROM table1

JOIN table2 ON table1.common_column = table2.common_column;

SUBQUERIES, VIEWS & STORED PROCEDURES



Subqueries

- A query is a statement used to retrieve data from a database, manipulate it, and return it in a formatted way.
- A subquery in SQL is a query nested inside another query. It is used to perform operations that require a value or set of values to be determined dynamically by another query.
- Subqueries can be used in various parts of a main SQL query, including the SELECT, FROM, WHERE and HAVING clauses.

SUBQUERIES, VIEWS & STORED PROCEDURES



Key aspects about subqueries are;

- **Return Value:** Subqueries can return individual values (scalar subqueries), a single column of values (column subqueries), or multiple rows and columns (table subqueries).
- **Placement:** Depending on the need, subqueries can appear in various parts of the main SQL query:
 - **WHERE Clause:** To filter results based on dynamic conditions.
 - **FROM Clause:** To define a temporary table for use in the main query.
 - **SELECT Clause:** To calculate values for each row returned by the main query.
- **Performance:** Subqueries can sometimes slow down query performance, especially if they are poorly written or if they are correlated subqueries that must be executed repeatedly.

SUBQUERIES, VIEWS & STORED PROCEDURES



- **Correlation:** Subqueries can be either correlated or uncorrelated. A **correlated subquery** refers back to fields from the outer query, thus requiring the outer query to execute and then the inner (subquery) for each row of the outer query. An **uncorrelated subquery** does not depend on the outer query and can be executed independently.
- The general syntax of a subquery varies based on its placement but its structure is as follows;

SELECT column_names

FROM table_name

WHERE expression operator (SELECT column_name FROM table_name WHERE condition);

SUBQUERIES, VIEWS & STORED PROCEDURES



Examples of subqueries with different placements:

```
SELECT Name, Age  
FROM Employees  
WHERE Age > (SELECT AVG(Age) FROM Employees);
```

```
SELECT avg_data.AverageAge  
FROM (SELECT AVG(Age) AS AverageAge FROM Employees) AS avg_data;
```

```
SELECT DepartmentID, AVG(Salary) AS AverageSalary  
FROM Employees  
GROUP BY DepartmentID  
HAVING AVG(Salary) > (SELECT AVG(Salary) FROM Employees);
```



SUBQUERIES, VIEWS & STORED PROCEDURES



```
SELECT DepartmentID, AVG(Salary) AS AverageSalary  
  
FROM Employees  
  
GROUP BY DepartmentID  
  
HAVING AVG(Salary) > (SELECT AVG(Salary) FROM Employees);
```

SUBQUERIES, VIEWS & STORED PROCEDURES



Views

- A **view** in SQL is a virtual table based on the result-set of an SQL statement. It contains rows and columns, just like a real table. The fields in a view are fields from one or more real tables in the database.
- Views can be used to;
- Simplify complex SQL queries by encapsulating them as views, making the underlying SQL simpler and more readable.
- Restrict access to data because a view can limit the user's ability to see certain columns or rows of the data.
- Present a different representation of data, aggregated, and computed columns can be simplified through views.

SUBQUERIES, VIEWS & STORED PROCEDURES



- The general syntax for creating a view is;

CREATE VIEW view_name AS

SELECT column1, column2, ...

FROM table_name

WHERE condition;

And to query the view created;

SELECT * FROM simple_customer_info;

SUBQUERIES, VIEWS & STORED PROCEDURES



- Views and temporary tables in SQL serve different purposes and have distinct characteristics, even though they might seem similar in that they both create a subset of data that can be worked with. Here's how they differ;

Views

- **Persistence:** Views are stored in the database schema and persist until explicitly dropped. They are essentially saved SQL queries.
- **Update Mechanism:** Views are virtual and are re-computed every time they are accessed. They dynamically reflect changes in the underlying tables.
- **Performance:** Because views are recalculated every time they are accessed, they might be slower for large datasets, unless materialized views are used (which are not supported by MySQL natively but can be emulated).
- **Usage:** Views are used to simplify complex queries, enforce security (by restricting access to specific columns or rows), and hide data complexity from users.
- **Memory:** Views do not occupy significant memory space as they only store the definition of the SQL query.

SUBQUERIES, VIEWS & STORED PROCEDURES



Temporary Tables

- **Persistence:** Temporary tables exist temporarily in the database server's memory or on disk. In MySQL, a temporary table is visible only within the current session, and is dropped automatically when the session is closed.
- **Update Mechanism:** Temporary tables are actual tables that can be indexed and stored in memory or on disk. You can perform any operations on them like with any regular table.
- **Performance:** Temporary tables can be faster for complex calculations that involve multiple SQL queries because the data is written and read from memory or disk, not recalculated on each access.
- **Usage:** Temporary tables are useful for storing intermediate results for complex calculations, especially in stored procedures or during complex transformation and loading (ETL) tasks.
- **Memory:** Temporary tables consume more memory because they temporarily store the data they hold.

SUBQUERIES, VIEWS & STORED PROCEDURES



Key Differences

- **Storage:** Views do not store data themselves—they are just a stored query. Temporary tables store data physically, which makes them faster at the cost of using more memory or disk space.
- **Recalculation vs. Persistence:** Views always present the latest data, reflecting changes to underlying tables. Temporary tables hold data as it was at the time it was inserted into the table, not changing unless explicitly updated.
- **Scope:** Views are accessible to any user with sufficient privileges, much like regular tables. Temporary tables are typically only accessible within the session they were created, providing a secure way to handle data without exposing it to all database users.

SUBQUERIES, VIEWS & STORED PROCEDURES



Stored Procedures

- Stored procedures are a powerful feature used in various database systems, including SQL Server, Oracle, MySQL, and PostgreSQL. They are essentially pre-written SQL code that you can save, so you can reuse and execute this code as needed. This can significantly boost productivity, ensure consistency, and improve security for database operations.

Key Features of Stored Procedures

- **Modularity:** Stored procedures allow encapsulation of SQL code into modules. This makes the database system more organized, as one can segment complex processes into manageable, specific tasks that are easier to maintain and understand.
- **Performance:** Once created, stored procedures are compiled and stored in the database. This pre-compilation helps in speeding up execution time since the database engine can reuse the execution plan without needing to parse and recompile the SQL code each time it's executed.

SUBQUERIES, VIEWS & STORED PROCEDURES



- **Efficiency in Network Traffic:** Using stored procedures can reduce network traffic between clients and servers because only calls to the procedures are sent over the network, rather than multiple lines of SQL code.
- **Security:** Stored procedures enhance security by abstracting the underlying data structures and granting access to users through procedure execution instead of direct access to data. You can control what data is accessed and modified by limiting operations within stored procedures.
- **Reusability and Maintainability:** Code encapsulation within stored procedures makes it easier to reuse code across different programs and applications without duplication. It also simplifies maintenance since changes made in the stored procedure are automatically reflected wherever it's used.

SUBQUERIES, VIEWS & STORED PROCEDURES



- The syntax for creating a stored procedure can vary between different database systems, but here's a general structure using MySQL workbench:

DELIMITER \$\$

CREATE PROCEDURE procedure_name (parameter_list)

BEGIN

-- SQL statements

END\$\$

DELIMITER ;

SUBQUERIES, VIEWS & STORED PROCEDURES



- DELIMITER: Changes the standard command end character(;) to something else, typically when defining stored procedures or triggers. It is necessary because these database objects often contain multiple SQL statements, and without changing the delimiter, MySQL would interpret each semicolon as the end of the command.
- CREATE PROCEDURE is the command used to define a new stored procedure.
- procedure_name: The name of your new procedure.
- parameter_list: this is the list of parameters the procedure will use. Each parameter can be marked as IN, OUT or INOUT
 - IN: A value passed to the procedure that can't be modified by it
 - OUT: A variable that the procedure can change to pass a value back to the caller
 - INOUT: A value passed into the procedure that can be modified and passed back.

SUBQUERIES, VIEWS & STORED PROCEDURES



- Procedure Body: The procedure body contains the SQL statements that define what the procedure will do, enclosed between BEGIN and END.

DELIMITER \$\$

```
CREATE PROCEDURE GetEmployeesByDepartment(IN DeptID INT)
```

```
BEGIN
```

```
    SELECT * FROM Employees WHERE DepartmentID = DeptID;
```

```
END$$
```

DELIMITER ;

-- To call the stored procedure:

```
CALL GetEmployeesByDepartment(3);
```

GROUP BY & ORDER BY



- GROUP BY is a clause used in SQL to group rows that have the same values in specified columns into summary rows, like find the number of customers in each country or calculate the average sales by region. It is often used with aggregate functions (COUNT, MAX, MIN, SUM, AVG) to perform the aggregation on the grouped rows.
- ORDER BY clause is used to sort the results of a query in either ascending or descending order. It can sort the data on one column or multiple columns and you can specify different sorting orders (ASC for ascending, DESC for descending) for each column.

Both of these clauses enhance the functionality of SQL queries by providing ways to aggregate, sort, and organize data more effectively based on specific conditions or criteria.

GROUP BY & ORDER BY



```
SELECT column_name(s), aggregate_function(column_name)
FROM table_name
WHERE condition
GROUP BY column_name(s);
```

EX:

```
SELECT Department_ID, COUNT(Employee_ID) AS NumberOfEmployees
FROM Employees
GROUP BY Department_ID;
```


GROUP BY & ORDER BY



```
SELECT column_name(s)
FROM table_name
WHERE condition
ORDER BY column_name(s) [ASC | DESC];
```

EX:

```
SELECT Name, Email
FROM Employees
ORDER BY Name ASC;
```

GROUP BY & ORDER BY



```
SELECT Department_ID, COUNT(Employee_ID) AS NumberOfEmployees  
FROM Employees  
GROUP BY Department_ID  
ORDER BY NumberOfEmployees DESC;
```

Ex:

```
SELECT Department_ID, COUNT(Employee_ID) AS NumberOfEmployees  
FROM Employees  
GROUP BY Department_ID  
ORDER BY NumberOfEmployees DESC;
```

HAVING CLAUSE



- The HAVING clause is used to filter records that work in summarized group data, returned by a GROUP BY clause. It's essentially a filter for aggregate functions, allowing conditions to be applied that determine which group results appear in the final output.

SELECT column_name(s), aggregate_function(column_name)

FROM table_name

WHERE condition

GROUP BY column_name(s)

HAVING condition_on_aggregate_function;

HAVING CLAUSE



Example:

```
SELECT Department_ID, COUNT(Employee_ID) AS NumberOfEmployees  
FROM Employees  
GROUP BY Department_ID  
HAVING COUNT(Employee_ID) > 10;
```